

РБНФ №1 (опис синтаксису всіма допустимими засобами РБНФ)	РБНФ №2 (опис формальної граматики засобами РБНФ)	Формальна граматика	Формальна граматика з специфікацією lookahead у правилах для LL(2)-аналізатора	Перевірка РБНФ №1 за допомогою коду /* Помістити у файл "EBNF_N1.h" */	Перевірка РБНФ №2 за допомогою коду /* Помістити у файл "EBNF_N2.h" */	Перевірки прототипу LL(2)-синтаксичного аналізатора (спеціальна структура) та прототипу лексичного аналізатора (регулярні вирази) за допомогою коду. Лексеми для синтаксичного аналізатора обробляються лексичним аналізатором, тому синтаксичний аналізатор не аналізує їх повторно (як показано в РБНФ). Помістити у файл "LexicaByRegExAndSyntaxByLL2prototype.h") УВАГА: при копіюванні зважайте, щоб у кожному рядку після символу «\» не містилось жодних інших символів. */
		G = (N, T, P, S)	G = (N, T, P, S)			
		S → program_rule	S → program_rule			
		N = { program_name, value_type, array_specify, declaration_element, array_specify_optional, other_declaration_ident, declaration, other_declaration_ident_iteration, index_action, unary_operator, unary_operation, binary_operator, binary_action, left_expression, group_expression, index_action_optional, expression, binary_action_iteration, expression_or_cond_block_with_optional_assign, assign_to_right, assign_to_right_optional, if_expression, body_for_true, false_cond_block_without_else, body_for_false, cond_block, false_cond_block_without_else_iteration, body_for_false_optional, statement, repeat_until_cycle_cond, repeat_until_cycle, statements_or_block_statements, block_statements, input_rule, argument_for_input, output_rule, statement_iteration, expression_optional, program_rule, declaration_optional, non_zero_digit, digit_iteration, digit, unsigned_value, value, sign_optional, sign, ident, letter_in_upper_case, letter_in_lower_case, sign_plus, sign_minus }	N = { program_name, value_type, array_specify, declaration_element, array_specify_optional, other_declaration_ident, declaration, other_declaration_ident_iteration, index_action, unary_operator, unary_operation, binary_operator, binary_action, left_expression, group_expression, index_action_optional, expression, binary_action_iteration, expression_or_cond_block_with_optional_assign, assign_to_right, assign_to_right_optional, if_expression, body_for_true, false_cond_block_without_else, body_for_false, cond_block, false_cond_block_without_else_iteration, body_for_false_optional, statement, repeat_until_cycle_cond, repeat_until_cycle, statements_or_block_statements, block_statements, input_rule, argument_for_input, output_rule, statement_iteration, expression_optional, program_rule, declaration_optional, non_zero_digit, digit_iteration, digit, unsigned_value, value, sign_optional, sign, ident, letter_in_upper_case, letter_in_lower_case, sign_plus, sign_minus }	#define NONTERMINALS program_name, \ value_type, \ array_specify, \ declaration_element, \ \ other_declaration_ident, \ declaration, \ \ index_action, \ unary_operator, \ unary_operation, \ binary_operator, \ binary_action, \ left_expression, \ group_expression, \ \ expression, \ \ expression_or_cond_block_with_optional_assign, \ assign_to_right, \ \ if_expression, \ body_for_true, \ false_cond_block_without_else, \ body_for_false, \ cond_block, \ \ statement, \ repeat_until_cycle_cond, \ repeat_until_cycle, \ statements_or_block_statements, \ block_statements, \ input_rule, \ argument_for_input, \ output_rule, \ \ \ program_rule, \ \ non_zero_digit, \ digit_iteration, \ digit, \ unsigned_value, \ value, \ \ sign, \ ident, \ letter_in_upper_case, \ letter_in_lower_case, \ sign_plus, \ sign_minus	#define NONTERMINALS program_name, \ value_type, \ array_specify, \ declaration_element, \ array_specify_optional, \ other_declaration_ident, \ declaration, \ other_declaration_ident_iteration, \ index_action, \ unary_operator, \ unary_operation, \ binary_operator, \ binary_action, \ left_expression, \ group_expression, \ index_action_optional, \ expression, \ binary_action_iteration, \ expression_or_cond_block_with_optional_assign, \ assign_to_right, \ assign_to_right_optional, \ if_expression, \ body_for_true, \ false_cond_block_without_else, \ body_for_false, \ cond_block, \ false_cond_block_without_else_iteration, \ body_for_false_optional, \ statement, \ repeat_until_cycle_cond, \ repeat_until_cycle, \ statements_or_block_statements, \ block_statements, \ input_rule, \ argument_for_input, \ output_rule, \ statement_iteration, \ expression_optional, \ program_rule, \ declaration_optional, \ non_zero_digit, \ digit_iteration, \ digit, \ unsigned_value, \ value, \ sign_optional, \ sign, \ ident, \ letter_in_upper_case, \ letter_in_lower_case, \ sign_plus, \ sign_minus	
		T = { "INTEGER16" , " " , "NOT" , "AND" , "OR" , "==" , "!=" , "<=" , ">=" , "+" , " " , "*" , "DIV" , "MOD" , "(" , ")" , ":", "ELSE" , "IF" , "EXIT" ,	T = { "INTEGER16" , " " , "NOT" , "AND" , "OR" , "==" , "!=" , "<=" , ">=" , "+" , " " , "*" , "DIV" , "MOD" , "(" , ")" , ":", "ELSE" , "IF" , "EXIT" ,	#define TOKENS \ tokenINTEGER16, \ tokenCOMMA, \ tokenNOT, \ tokenAND, \ tokenOR, \ tokenEQUAL, \ tokenNOTEQUAL, \ tokenLESSOREQUAL, \ tokenGREATEROREQUAL, \ tokenPLUS, \ tokenMINUS, \ tokenMUL, \ tokenDIV, \ tokenMOD, \ tokenGROUPEXPRESSIONBEGIN, \ tokenGROUPEXPRESSIONEND, \ tokenLRASSIGN, \ tokenELSE, \ tokenIF, \ tokenEXIT, \	#define TOKENS \ tokenINTEGER16, \ tokenCOMMA, \ tokenNOT, \ tokenAND, \ tokenOR, \ tokenEQUAL, \ tokenNOTEQUAL, \ tokenLESSOREQUAL, \ tokenGREATEROREQUAL, \ tokenPLUS, \ tokenMINUS, \ tokenMUL, \ tokenDIV, \ tokenMOD, \ tokenGROUPEXPRESSIONBEGIN, \ tokenGROUPEXPRESSIONEND, \ tokenLRASSIGN, \ tokenELSE, \ tokenIF, \ tokenEXIT, \	

						#define T_END_BLOCK_3 ""
				tokenSEMICOLON = ";" >> BOUNDARIES;	tokenSEMICOLON = ";" >> BOUNDARIES;	#define T_SEMICOLON_0 ";" #define T_SEMICOLON_1 "" #define T_SEMICOLON_2 "" #define T_SEMICOLON_3 ""
				tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;	tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;	#define T_DATA_TYPE_0 "INTEGER16" #define T_DATA_TYPE_1 "" #define T_DATA_TYPE_2 "" #define T_DATA_TYPE_3 ""
				tokenCOMMA = "," >> BOUNDARIES;	tokenCOMMA = "," >> BOUNDARIES;	#define T_COMA_0 "," #define T_COMA_1 "" #define T_COMA_2 "" #define T_COMA_3 ""
						#define T_BITWISE_NOT_0 "~" #define T_BITWISE_NOT_1 "" #define T_BITWISE_NOT_2 "" #define T_BITWISE_NOT_3 ""
				tokenNOT = "NOT" >> STRICT_BOUNDARIES;	tokenNOT = "NOT" >> STRICT_BOUNDARIES;	#define T_NOT_0 "NOT" #define T_NOT_1 "" #define T_NOT_2 "" #define T_NOT_3 ""
						#define T_BITWISE_AND_0 "&" #define T_BITWISE_AND_1 "" #define T_BITWISE_AND_2 "" #define T_BITWISE_AND_3 ""
				tokenAND = "AND" >> STRICT_BOUNDARIES;	tokenAND = "AND" >> STRICT_BOUNDARIES;	#define T_AND_0 "AND" #define T_AND_1 "" #define T_AND_2 "" #define T_AND_3 ""
						#define T_BITWISE_OR_0 " " #define T_BITWISE_OR_1 "" #define T_BITWISE_OR_2 "" #define T_BITWISE_OR_3 ""
				tokenOR = "OR" >> STRICT_BOUNDARIES;	tokenOR = "OR" >> STRICT_BOUNDARIES;	#define T_OR_0 "OR" #define T_OR_1 "" #define T_OR_2 "" #define T_OR_3 ""
				tokenEQUAL = "==" >> BOUNDARIES;	tokenEQUAL = "==" >> BOUNDARIES;	#define T_EQUAL_0 "==" #define T_EQUAL_1 "" #define T_EQUAL_2 "" #define T_EQUAL_3 ""
				tokenNOTEQUAL = "!=" >> BOUNDARIES;	tokenNOTEQUAL = "!=" >> BOUNDARIES;	#define T_NOT_EQUAL_0 "!=" #define T_NOT_EQUAL_1 "" #define T_NOT_EQUAL_2 "" #define T_NOT_EQUAL_3 ""
				tokenLESSOREQUAL = "<=" >> BOUNDARIES;	tokenLESSOREQUAL = "<=" >> BOUNDARIES;	#define T_LESS_OR_EQUAL_0 "<="
				tokenGREATEROREQUAL = ">=" >> BOUNDARIES;	tokenGREATEROREQUAL = ">=" >> BOUNDARIES;	#define T_GREATER_OR_EQUAL_0 ">="
				tokenPLUS = "+" >> BOUNDARIES;	tokenPLUS = "+" >> BOUNDARIES;	#define T_ADD_0 "+" #define T_ADD_1 "" #define T_ADD_2 "" #define T_ADD_3 ""
				tokenMINUS = "-" >> BOUNDARIES;	tokenMINUS = "-" >> BOUNDARIES;	#define T_SUB_0 "-" #define T_SUB_1 "" #define T_SUB_2 "" #define T_SUB_3 ""
				tokenMUL = "*" >> BOUNDARIES;	tokenMUL = "*" >> BOUNDARIES;	#define T_MUL_0 "*"
				tokenDIV = "DIV" >> STRICT_BOUNDARIES;	tokenDIV = "DIV" >> STRICT_BOUNDARIES;	#define T_DIV_0 "DIV" #define T_DIV_1 "" #define T_DIV_2 "" #define T_DIV_3 ""
				tokenMOD = "MOD" >> STRICT_BOUNDARIES;	tokenMOD = "MOD" >> STRICT_BOUNDARIES;	#define T_MOD_0 "MOD" #define T_MOD_1 "" #define T_MOD_2 "" #define T_MOD_3 ""
				tokenLRASSIGN = "=" >> BOUNDARIES;	tokenLRASSIGN = "=" >> BOUNDARIES;	#define T_LRASSIGN_0 "=" #define T_LRASSIGN_1 "" #define T_LRASSIGN_2 "" #define T_LRASSIGN_3 ""
						#define T_THEN_BLOCK_0 "(" #define T_THEN_BLOCK_1 "" #define T_THEN_BLOCK_2 "" #define T_THEN_BLOCK_3 ""
				tokenELSE = "ELSE" >> STRICT_BOUNDARIES;	tokenELSE = "ELSE" >> STRICT_BOUNDARIES;	#define T_ELSE_BLOCK_0 "ELSE" #define T_ELSE_BLOCK_1 T_BEGIN_BLOCK_0 #define T_ELSE_BLOCK_2 "" #define T_ELSE_BLOCK_3 ""
				tokenIF = "IF" >> STRICT_BOUNDARIES;	tokenIF = "IF" >> STRICT_BOUNDARIES;	#define T_IF_0 "IF" #define T_IF_1 "" #define T_IF_2 "" #define T_IF_3 ""
						#define T_ELSE_IF_0 "ELSE" #define T_ELSE_IF_1 T_IF_0 #define T_ELSE_IF_2 "" #define T_ELSE_IF_3 ""
				tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;	tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;	#define T_EXIT_0 "EXIT" #define T_EXIT_1 "" #define T_EXIT_2 "" #define T_EXIT_3 ""
				tokenREPEAT = "REPEAT" >> STRICT_BOUNDARIES;	tokenREPEAT = "REPEAT" >> STRICT_BOUNDARIES;	#define T_REPEAT_0 "REPEAT" #define T_REPEAT_1 "" #define T_REPEAT_2 "" #define T_REPEAT_3 ""
				tokenUNTIL = "UNTIL" >> STRICT_BOUNDARIES;	tokenUNTIL = "UNTIL" >> STRICT_BOUNDARIES;	#define T_UNTIL_0 "UNTIL" #define T_UNTIL_1 "" #define T_UNTIL_2 "" #define T_UNTIL_3 ""
				tokenGET = "GET" >> STRICT_BOUNDARIES;	tokenGET = "GET" >> STRICT_BOUNDARIES;	#define T_INPUT_0 "GET"

						#define T_INPUT_1 "" #define T_INPUT_2 "" #define T_INPUT_3 ""
				tokenPUT = "PUT" >> STRICT_BOUNDARIES;	tokenPUT = "PUT" >> STRICT_BOUNDARIES;	#define T_OUTPUT_0 "PUT" #define T_OUTPUT_1 "" #define T_OUTPUT_2 "" #define T_OUTPUT_3 ""
				tokenNAME = "NAME" >> STRICT_BOUNDARIES;	tokenNAME = "NAME" >> STRICT_BOUNDARIES;	#define T_NAME_0 "NAME" #define T_NAME_1 "" #define T_NAME_2 "" #define T_NAME_3 ""
				tokenBODY = "BODY" >> STRICT_BOUNDARIES;	tokenBODY = "BODY" >> STRICT_BOUNDARIES;	#define T_BODY_0 "BODY" #define T_BODY_1 "" #define T_BODY_2 "" #define T_BODY_3 ""
				tokenDATA = "DATA" >> STRICT_BOUNDARIES;	tokenDATA = "DATA" >> STRICT_BOUNDARIES;	#define T_DATA_0 "DATA" #define T_DATA_1 "" #define T_DATA_2 "" #define T_DATA_3 ""
				tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;	tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;	#define T_BEGIN_0 "BEGIN" #define T_BEGIN_1 "" #define T_BEGIN_2 "" #define T_BEGIN_3 ""
				tokenEND = "END" >> STRICT_BOUNDARIES;	tokenEND = "END" >> STRICT_BOUNDARIES;	#define T_END_0 "END" #define T_END_1 "" #define T_END_2 "" #define T_END_3 ""
						#define T_NULL_STATEMENT_0 "NULL" #define T_NULL_STATEMENT_1 "STATEMENT" #define T_NULL_STATEMENT_2 "" #define T_NULL_STATEMENT_3 ""
						#define GRAMMAR_LL2_2025 {\
program_name = ident;	program_name = ident;	program_name → ident	program_name(1: "ident_terminal") → ident	program_name = SAME_RULE(ident);	program_name = SAME_RULE(ident);	{ LA_IS, ("ident_terminal"), { "program_name", {\
value_type = "INTEGER16";	value_type = "INTEGER16";	value_type → "INTEGER16"	value_type(1: "INTEGER16") → "INTEGER16"	value_type = SAME_RULE(tokenINTEGER16);	value_type = SAME_RULE(tokenINTEGER16);	{ LA_IS, (T_DATA_TYPE_0), { "value_type", {\
	array_specify = "[", unsigned_value, "]"	array_specify → "[" unsigned_value "]"	array_specify(1: "["]") → "[" unsigned_value "]"		array_specify = "[" >> unsigned_value >> "]"	{ LA_IS, ("["], { "array_specify", {\
declaration_element = ident, array_specify__optional	declaration_element = ident, array_specify__optional	declaration_element → ident array_specify__optional	declaration_element(1: "ident_terminal") → ident array_specify__optional	declaration_element = ident >> -(tokenLEFTSQUAREBRACKETS >> unsigned_value >> tokenRIGHTSQUAREBRACKETS);	declaration_element = ident >> array_specify__optional;	{ LA_IS, ("ident_terminal"), { "declaration_element", {\
	array_specify__optional = array_specify ε	array_specify__optional → array_specify array_specify__optional → ε	array_specify__optional(1: "["]") → array_specify array_specify__optional(1: ! "["]") → ε		array_specify__optional = array_specify ""	{ LA_IS, ("["], { "array_specify__optional", {\
other_declaration_ident = " ,", declaration_element	other_declaration_ident = " ,", declaration_element	other_declaration_ident → " ,", declaration_element	other_declaration_ident(1: " ,",) → " ,", declaration_element	other_declaration_ident = tokenCOMMA >> declaration_element	other_declaration_ident = tokenCOMMA >> declaration_element	{ LA_IS, (T_COMA_0), { "other_declaration_ident", {\
declaration = value_type , declaration_element , {other_declaration_ident};	declaration = value_type , declaration_element , other_declaration_ident__iteration	declaration → value_type declaration_element other_declaration_ident__iteration	declaration(1: "INTEGER16") → value_type declaration_element other_declaration_ident__iteration	declaration = value_type >> declaration_element >> *other_declaration_ident	declaration = value_type >> declaration_element >> other_declaration_ident__iteration	{ LA_IS, (T_DATA_TYPE_0), { "declaration", {\
	other_declaration_ident__iteration = other_declaration_ident, other_declaration_ident__iteration ε	other_declaration_ident__iteration → other_declaration_ident other_declaration_ident__iteration false_cond_block_without_else__iteration → ε	other_declaration_ident__iteration(1: " ,",) → other_declaration_ident other_declaration_ident__iteration false_cond_block_without_else__iteration(1: ! " ,",) → ε		other_declaration_ident__iteration = other_declaration_ident >> other_declaration_ident__iteration ""	{ LA_IS, (T_COMA_0), { "other_declaration_ident__iteration", {\
index_action = "[" , expression , "]"	index_action = "[" , expression , "]"	index_action → "[" expression "]"	index_action(1: "["]") → "[" expression "]"	index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS	index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS	{ LA_IS, ("["], { "index_action", {\
unary_operator = "NOT"	unary_operator = "NOT"	unary_operator → "NOT"	unary_operator(1: "NOT") → "NOT"	unary_operator = SAME_RULE(tokenNOT)	unary_operator = SAME_RULE(tokenNOT)	{ LA_IS, (T_NOT_0), { "unary_operator", {\
unary_operation = unary_operator , expression	unary_operation = unary_operator , expression	unary_operation → unary_operator expression	unary_operation(1: "NOT") → unary_operator expression	unary_operation = unary_operator >> expression	unary_operation = unary_operator >> expression	{ LA_IS, (T_NOT_0), { "unary_operation", {\
binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "+" "-" "*" "DIV" "MOD"	binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "+" "-" "*" "DIV" "MOD"	binary_operator → "AND" binary_operator → "OR" binary_operator → "==" binary_operator → "!=" binary_operator → "<=" binary_operator → ">=" binary_operator → "+" binary_operator → "-" binary_operator → "*" binary_operator → "DIV" binary_operator → "MOD"	binary_operator(1: "AND") → "AND" binary_operator(1: "OR") → "OR" binary_operator(1: "==") → "==" binary_operator(1: "!=") → "!=" binary_operator(1: "<=") → "<=" binary_operator(1: ">=") → ">=" binary_operator(1: "+") → "+" binary_operator(1: "-") → "-" binary_operator(1: "*") → "*" binary_operator(1: "DIV") → "DIV" binary_operator(1: "MOD") → "MOD"	binary_operator = tokenAND tokenOR tokenEQUAL tokenNOTEQUAL tokenLESSOREQUAL tokenGREATEROREQUAL tokenPLUS tokenMINUS tokenMUL tokenDIV tokenMOD	binary_operator = tokenAND tokenOR tokenEQUAL tokenNOTEQUAL tokenLESSOREQUAL tokenGREATEROREQUAL tokenPLUS tokenMINUS tokenMUL tokenDIV tokenMOD	{ LA_IS, (T_AND_0), { "binary_operator", {\

						{ LA_IS, {""}, 1, { T_MOD_0 }}\n}};\n
binary_action = binary_operator , expression;	binary_action = binary_operator , expression;	binary_action → binary_operator expression	binary_action(1: "AND", "OR", "=", "!=", "<=", ">=", "+", "-", "**", "DIV", "MOD") → binary_operator expression	binary_action = binary_operator >> expression;	binary_action = binary_operator >> expression;	{ LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action", {\n{ LA_IS, {""}, 2, { "binary_operator", "expression" }}\n}};\n
left_expression = group_expression unary_operation ident , [index_action] value cond_block;	left_expression = group_expression unary_operation ident , index_action__optional value cond_block;	left_expression → group_expression left_expression → unary_operation left_expression → ident , index_action__optional left_expression → value left_expression → cond_block	left_expression(1: "(") → group_expression left_expression(1: "NOT") → unary_operation left_expression(1: "-") → ident , index_action__optional left_expression(1: "0", "1", "2", "3", "4", "5", "6", "7", "8", "9") → value left_expression(1: "+", "-", "2: "0", "1", "2", "3", "4", "5", "6", "7", "8", "9") → value left_expression(1: "IF") → cond_block	left_expression = group_expression unary_operation ident >> -index_action value cond_block;	left_expression = group_expression unary_operation ident >> index_action__optional value cond_block;	{LA_IS, { "(" }, { "left_expression", {\n{ LA_IS, { "" }, 1, { "group_expression" }}\n}};\n{LA_IS, { T_NOT_0 }, { "left_expression", {\n{ LA_IS, { "" }, 1, { "unary_operation" }}\n}};\n{LA_IS, { "ident_terminal" }, { "left_expression", {\n{ LA_IS, { "" }, 2, { "ident", "index_action__optional" }}\n}};\n{LA_IS, { "unsigned_value_terminal" }, { "left_expression", {\n{ LA_IS, { "" }, 1, { "value" }}\n}};\n{LA_IS, { T_ADD_0, T_SUB_0 }, { "left_expression", {\n{ LA_IS, { "unsigned_value_terminal" }, 1, { "value" }}},\n/*{LA_NOT, { "unsigned_value_terminal" }, 1, { "unary_operation" }}*\n}};\n{LA_IS, { T_IF_0 }, { "left_expression", {\n{ LA_IS, { "" }, 1, { "cond_block" }}\n}};\n
index_action__optional = index_action ε;	index_action__optional = index_action ε;	index_action__optional → index_action index_action__optional → ε	index_action__optional(1: "!") → index_action index_action__optional(1: "!") → ε	index_action__optional = index_action "";	index_action__optional = index_action "";	{LA_IS, { "" }, { "index_action__optional", {\n{ LA_IS, { "" }, 1, { "index_action" }}\n}};\n{LA_NOT, { "" }, { "index_action__optional", {\n{ LA_IS, { "" }, 0, { "" }}\n}};\n
expression = left_expression , binary_action__iteration;	expression = left_expression , binary_action__iteration;	expression → left_expression binary_action__iteration	expression(1: "(", "NOT", "+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → left_expression binary_action__iteration	expression = left_expression >> *binary_action;	expression = left_expression >> binary_action__iteration;	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, { "expression", {\n{ LA_IS, { "" }, 2, { "left_expression", "binary_action__iteration" }}\n}};\n
binary_action__iteration = binary_action, binary_action__iteration ε;	binary_action__iteration = binary_action, binary_action__iteration ε;	binary_action__iteration → binary_action binary_action__iteration binary_action__iteration → ε	binary_action__iteration(1: "AND", "OR", "=", "!=", "<=", ">=", "+", "-", "**", "DIV", "MOD") → binary_action binary_action__iteration binary_action__iteration(1: "AND", "OR", "I"=, "I"!=, "I"<=", "I">=", "I"+, "I"-, "I**", "I"DIV", "I"MOD") → ε	binary_action__iteration = binary_action >> binary_action__iteration "";	binary_action__iteration = binary_action >> binary_action__iteration "";	{LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {\n{ LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" }}\n}};\n{LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {\n{ LA_IS, { "" }, 0, { "" }}\n}};\n
group_expression = "(" , expression , ")";	group_expression = "(" , expression , ")";	group_expression → "(" expression ")"	group_expression(1: "(") → "(" expression ")"	group_expression = tokenGROUPEXPRESSSIONBEGIN >> expression >> tokenGROUPEXPRESSSIONEND;	group_expression = tokenGROUPEXPRESSSIONBEGIN >> expression >> tokenGROUPEXPRESSSIONEND;	{LA_IS, { "(" }, { "group_expression", {\n{ LA_IS, { "" }, 3, { "(", "expression", ")" }}\n}};\n
expression_or_cond_block__with_optional_assign = expression , assign_to_right__optional;	expression_or_cond_block__with_optional_assign = expression , assign_to_right__optional;	expression_or_cond_block__with_optional_assign → expression assign_to_right__optional	expression_or_cond_block__with_optional_assign(1: "(", "NOT", "+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression assign_to_right__optional	expression_or_cond_block__with_optional_assign = expression >> -(tokenLRASSIGN >> ident >> -index_action);	expression_or_cond_block__with_optional_assign = expression >> assign_to_right__optional;	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, { "expression_or_cond_block__with_optional_assign", {\n{ LA_IS, { "" }, 2, { "expression", "assign_to_right__optional" }}\n}};\n
assign_to_right = "=: " , ident , index_action__optional;	assign_to_right = "=: " , ident , index_action__optional;	assign_to_right → "=: " ident index_action__optional	assign_to_right(1: "=: ") → "=: " ident index_action__optional	assign_to_right = tokenLRASSIGN >> ident >> index_action__optional;	assign_to_right = tokenLRASSIGN >> ident >> index_action__optional;	{LA_IS, { T_LRASSIGN_0 }, { "assign_to_right", {\n{ LA_IS, { "" }, 3, { T_LRASSIGN_0, "ident", "index_action__optional" }}\n}};\n
assign_to_right__optional = assign_to_right ε;	assign_to_right__optional = assign_to_right ε;	assign_to_right__optional → assign_to_right assign_to_right__optional → ε;	assign_to_right__optional(1: "=: ") → assign_to_right assign_to_right__optional(1: "=: ") → ε;	assign_to_right__optional = assign_to_right "";	assign_to_right__optional = assign_to_right "";	{ LA_IS, { T_LRASSIGN_0 }, { "assign_to_right__optional", {\n{ LA_IS, { "" }, 1, { "assign_to_right" }}\n}};\n{ LA_NOT, { T_LRASSIGN_0 }, { "assign_to_right__optional", {\n{ LA_IS, { "" }, 0, { "" }}\n}};\n
if_expression = expression;	if_expression = expression;	if_expression → expression	if_expression(1: "(", "NOT", "+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression	if_expression = SAME_RULE(expression);	if_expression = SAME_RULE(expression);	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, { "if_expression", {\n{ LA_IS, { "" }, 1, { "expression" }}\n}};\n
body_for_true = block_statements;	body_for_true = block_statements;	body_for_true → block_statements	body_for_true(1: "(") → block_statements	body_for_true = SAME_RULE(block_statements);	body_for_true = SAME_RULE(block_statements);	{LA_IS, { T_BEGIN_BLOCK_0 }, { "body_for_true", {\n{ LA_IS, { "" }, 1, { "block_statements" }}\n}};\n
false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true;	false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true;	false_cond_block_without_else → "ELSE" "IF" if_expression body_for_true	false_cond_block_without_else(1: "ELSE") → "ELSE" "IF" if_expression body_for_true	false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true;	false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true;	{LA_IS, { T_ELSE_IF_0 }, { "false_cond_block_without_else", {\n{ LA_IS, { "" }, 4, { T_ELSE_IF_0, T_ELSE_IF_1, "if_expression", "body_for_true" }}\n}};\n
body_for_false = "ELSE" , block_statements;	body_for_false = "ELSE" , block_statements;	body_for_false → "ELSE" block_statements	body_for_false(1: "ELSE") → "ELSE" block_statements	body_for_false = tokenELSE >> block_statements;	body_for_false = tokenELSE >> block_statements;	{LA_IS, { T_ELSE_BLOCK_0 }, { "body_for_false", {\n{ LA_IS, { "" }, 2, { T_ELSE_BLOCK_0, "block_statements" }}\n}};\n
cond_block = "IF" , if_expression , body_for_true, false_cond_block_without_else__iteration , body_for_false__optional;	cond_block = "IF" , if_expression , body_for_true, false_cond_block_without_else__iteration , body_for_false__optional;	cond_block → "IF" if_expression body_for_true false_cond_block_without_else__iteration body_for_false__optional	cond_block(1: "IF") → "IF" if_expression body_for_true false_cond_block_without_else__iteration body_for_false__optional	cond_block = tokenIF >> if_expression >> body_for_true >> *false_cond_block_without_else >> -body_for_false;	cond_block = tokenIF >> if_expression >> body_for_true >> false_cond_block_without_else__iteration >> body_for_false__optional;	{LA_IS, { T_IF_0 }, { "cond_block", {\n{ LA_IS, { "" }, 5, { T_IF_0, "if_expression", "body_for_true", "false_cond_block_without_else__iteration", "body_for_false__optional" }}\n}};\n
false_cond_block_without_else__iteration = false_cond_block_without_else, false_cond_block_without_else__iteration ε;	false_cond_block_without_else__iteration = false_cond_block_without_else, false_cond_block_without_else__iteration ε;	false_cond_block_without_else__iteration → false_cond_block_without_else false_cond_block_without_else__iteration false_cond_block_without_else__iteration → ε	false_cond_block_without_else__iteration(1: "ELSE"; 2: "IF") → false_cond_block_without_else false_cond_block_without_else__iteration(1: "ELSE"; 2: "IF") → ε false_cond_block_without_else__iteration(1: "ELSE") → ε	false_cond_block_without_else__iteration = false_cond_block_without_else >> false_cond_block_without_else__iteration "";	false_cond_block_without_else__iteration = false_cond_block_without_else >> false_cond_block_without_else__iteration "";	{LA_IS, { T_ELSE_IF_0 }, { "false_cond_block_without_else__iteration", {\n{ LA_IS, { T_ELSE_IF_1 }, 2, { "false_cond_block_without_else", "false_cond_block_without_else__iteration" }}},\n{LA_NOT, { T_ELSE_IF_1 }, 0, { "" }}\n}};\n

						<pre>{LA_NOT,{T_ELSE_IF_0},{ "false_cond_block_without_else_iteration",{\ {LA_IS,(""),0,{ "" }}\ }}\}</pre>
	<code>body_for_false__optional = body_for_false ε;</code>	<code>body_for_false__optional → body_for_false body_for_false__optional → ε</code>	<code>body_for_false__optional(1: "FALSE") → body_for_false body_for_false__optional(1: !"FALSE") → ε</code>		<code>body_for_false__optional = body_for_false "";</code>	<pre>{LA_IS,{T_ELSE_BLOCK_0},{ "body_for_false_optional",{\ {LA_IS,(""),1,{ "body_for_false" }}\ }}\} {LA_NOT,{T_ELSE_BLOCK_0},{ "body_for_false_optional",{\ {LA_IS,(""),0,{ "" }}\ }}\}</pre>
<code>repeat_until_cycle_cond = expression;</code>	<code>repeat_until_cycle_cond = expression;</code>	<code>repeat_until_cycle_cond → expression</code>	<code>repeat_until_cycle_cond(1: "(","NOT","+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression</code>		<code>repeat_until_cycle_cond = SAME_RULE(expression);</code>	<pre>{LA_IS,{T_NOT_0,T_ADD_0,T_SUB_0, "ident_terminal", "unsigned_value_terminal",T_IF_0}, { "repeat_until_cycle_cond",{\ {LA_IS,(""),1,{ "expression" }}\ }}\}</pre>
<code>repeat_until_cycle = "REPEAT" , ({statement} block_statements) , "UNTIL" , repeat_until_cycle_cond;</code>	<code>repeat_until_cycle = "REPEAT" statements__or__block_statements , "UNTIL" , repeat_until_cycle_cond;</code>	<code>repeat_until_cycle → "REPEAT" statements__or__block_statements "UNTIL" repeat_until_cycle_cond</code>	<code>repeat_until_cycle(1: "REPEAT") → "REPEAT" statements__or__block_statements "UNTIL" repeat_until_cycle_cond</code>	<code>repeat_until_cycle = tokenREPEAT >> statements__or__block_statements >> tokenUNTIL >> repeat_until_cycle_cond;</code>	<code>repeat_until_cycle = tokenREPEAT >> statements__or__block_statements >> tokenUNTIL >> repeat_until_cycle_cond;</code>	<pre>{LA_IS,{T_REPEAT_0},{ "repeat_until_cycle",{\ {LA_IS,(""),4,{T_REPEAT_0, "statements__or__block_statements",T_UNTIL_0, "repeat_until_cycle_cond" }}\ }}\}</pre>
	<code>statements__or__block_statements = statement__iteration block_statements;</code>	<code>statements__or__block_statements → statement__iteration statements__or__block_statements → block_statements</code>	<code>statements__or__block_statements(1: " ", "(" , "NOT", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "+", "-", "IF", "FOR", "REPEAT", "GOTO", "GET", "PUT", ";") → statement__iteration statements__or__block_statements(1: "(") → block_statements</code>		<code>statements__or__block_statements = statement__iteration block_statements;</code>	<pre>{LA_NOT,{T_REPEAT_0},{ "statements__or__block_statements",{\ {LA_IS,(""),1,{ "statement__iteration" }}\ }}\} {LA_IS,{T_BEGIN_BLOCK_0},{ "statements__or__block_statements",{\ {LA_IS,(""),1,{ "block_statements" }}\ }}\}</pre>
<code>input_rule = "GET" , (ident , [index_action] "(" , ident , [index_action] , ")");</code>	<code>input_rule = "GET" , argument_for_input;</code>	<code>input_rule → "GET" argument_for_input</code>	<code>input_rule(1: "GET") → "GET" argument_for_input</code>	<code>input_rule = tokenGET >> (ident >> -index_action tokenGROUPEXPRESSIONBEGIN >> ident >> -index_action >> tokenGROUPEXPRESSIONEND);</code>	<code>input_rule = tokenGET >> argument_for_input;</code>	<pre>{LA_IS,{T_INPUT_0},{ "input_rule",{\ {LA_IS,(""),2,{T_INPUT_0,"argument_for_input" }}\ }}\}</pre>
	<code>argument_for_input = ident , index_action__optional; argument_for_input = "(" , "ident", "index_action__optional", ")" ;</code>	<code>argument_for_input → ident index_action__optional argument_for_input → "(" "ident" "index_action__optional" ")"</code>	<code>argument_for_input(1: " ") → ident index_action__optional argument_for_input(1: "(") → "(" "ident" "index_action__optional" ")"</code>		<code>argument_for_input = ident >> index_action__optional tokenGROUPEXPRESSIONBEGIN >> ident >> index_action__optional >> tokenGROUPEXPRESSIONEND;</code>	<pre>{LA_IS,{ "ident_terminal" },{ "argument_for_input",{\ {LA_IS,(""),2,{ "ident", "index_action__optional" }}\ }}\} {LA_IS,{ " " },{ "argument_for_input",{\ {LA_IS,(""),4,{ "(", "ident", "index_action__optional", ")" }}\ }}\}</pre>
<code>output_rule = "PUT" , expression;</code>	<code>output_rule = "PUT" , expression;</code>	<code>output_rule → "PUT" expression</code>	<code>output(1: "PUT") → "PUT" expression</code>	<code>output_rule = tokenPUT >> expression;</code>	<code>output_rule = tokenPUT >> expression;</code>	<pre>{LA_IS,{T_OUTPUT_0},{ "output_rule",{\ {LA_IS,(""),2,{T_OUTPUT_0,"expression" }}\ }}\}</pre>
<code>statement = expression_or_cond_block__with_optional_assign repeat_until_cycle input_rule output_rule output_rule ";" ;</code>	<code>statement = expression_or_cond_block__with_optional_assign repeat_until_cycle input_rule output_rule ";" ;</code>	<code>statement → expression_or_cond_block__with_optional_assign statement → repeat_until_cycle statement → input_rule statement → output_rule statement → ";"</code>	<code>statement(1: "(" , "NOT", "+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression_or_cond_block__with_optional_assign statement(1: "REPEAT") → repeat_until_cycle statement(1: "GET") → input_rule statement(1: "PUT") → output_rule statement(1: ";") → ";"</code>	<code>statement = expression_or_cond_block__with_optional_assign repeat_until_cycle input_rule output_rule tokenSEMICOLON;</code>	<code>statement = expression_or_cond_block__with_optional_assign repeat_until_cycle input_rule output_rule tokenSEMICOLON;</code>	<pre>{LA_IS,{T_NOT_0, "ident_terminal", "unsigned_value_terminal",T_ADD_0,T_SUB_0,T_IF_0}, { "statement", {\ {LA_IS,(""),1, {("expression_or_cond_block__with_optional_assign")}}\ }}\} {LA_IS,{T_REPEAT_0},{ "statement",{\ {LA_IS,(""),1,{ "repeat_until_cycle" }}\ }}\} {LA_IS,{T_INPUT_0},{ "statement",{\ {LA_IS,(""),1,{ "input_rule" }}\ }}\} {LA_IS,{T_OUTPUT_0},{ "statement",{\ {LA_IS,(""),1,{ "output_rule" }}\ }}\} {LA_IS,{T_SEMICOLON_0},{ "statement",{\ {LA_IS,(""),1,{ ";" }}\ }}\}</pre>
	<code>statement__iteration = statement, statement__iteration ε;</code>	<code>statement__iteration → statement statement__iteration statement__iteration → ε</code>	<code>statement__iteration(1: " ", "(" , "NOT", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "+", "-", "IF", "FOR", "REPEAT", "GOTO", "GET", "PUT", ";") → statement statement__iteration statement__iteration(1: !" ", !" "(" , !"NOT", !"0", !"1", !"2", !"3", !"4", !"5", !"6", !"7", !"8", !"9", !" +", !"IF", !"FOR", !"REPEAT", !"GOTO", !"GET", !"PUT", !";") → ε</code>		<code>statement__iteration = statement >> statement__iteration "";</code>	<pre>{LA_IS,{ "ident_terminal", "(" ,T_NOT_0, "unsigned_value_terminal",T_ADD_0,T_SUB_0, T_IF_0,T_REPEAT_0,T_INPUT_0,T_OUTPUT_0, T_SEMICOLON_0},{ "statement__iteration",{\ {LA_IS,(""),2,{ "statement", "statement__iteration" }}\ }}\} {LA_NOT,{ "ident_terminal", "(" ,T_NOT_0, "unsigned_value_terminal",T_ADD_0,T_SUB_0, T_IF_0,T_REPEAT_0,T_INPUT_0,T_OUTPUT_0, T_SEMICOLON_0},{ "statement__iteration",{\ {LA_IS,(""),0,{ "" }}\ }}\}</pre>
<code>block_statements = "{" , {statement} , "}" ;</code>	<code>block_statements = "{" , statement__iteration , "}" ;</code>	<code>block_statements → "{" statement__iteration" }</code>	<code>block_statements(1: "{") → "{" statement__iteration "}"</code>	<code>block_statements = tokenBEGINBLOCK >> statement__iteration >> tokenENDBLOCK;</code>	<code>block_statements = tokenBEGINBLOCK >> statement__iteration >> tokenENDBLOCK;</code>	<pre>{LA_IS,{T_BEGIN_BLOCK_0},{ "block_statements",{\ {LA_IS,(""),3,{T_BEGIN_BLOCK_0, "statement__iteration",T_END_BLOCK_0}}\ }}\}</pre>
	<code>expression__optional = expression "";</code>	<code>expression__optional → expression expression__optional → ε</code>	<code>expression__optional(1: "(" , "NOT", "+", "-", " ", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression expression__optional(1: !" "(" , !"NOT", !" +", !" -, !" " !"0", !"1", !"2", !"3", !"4", !"5", !"6", !"7", !"8", !"9", !"IF") → ε</code>		<code>expression__optional = expression "";</code>	<pre>{LA_IS,{ "(" ,T_NOT_0,T_ADD_0,T_SUB_0, "ident_terminal", "unsigned_value_terminal",T_IF_0}, { "expression__optional",{\ {LA_IS,(""),1,{ "expression" }}\ }}\} {LA_NOT,{ "(" ,T_NOT_0,T_ADD_0,T_SUB_0, "ident_terminal", "unsigned_value_terminal",T_IF_0}, { "expression__optional",{\ {LA_IS,(""),0,{ "" }}\ }}\}</pre>
<code>program_rule = "NAME" , program_name , ";" , "BODY" , "DATA" , declaration__optional , ";" , statement__iteration , "END" ;</code>	<code>program_rule = "NAME" , program_name , ";" , "BODY" , "DATA" , declaration__optional , ";" , statement__iteration , "END" ;</code>	<code>program_rule → "NAME" program_name ";" "BODY" "DATA" declaration__optional ";" statement__iteration "END"</code>	<code>program_rule(1: "NAME") → "NAME" program_name ";" "BODY" "DATA" declaration__optional ";" statement__iteration "END"</code>	<code>program_rule = BOUNDARIES >> tokenNAME >> program_name >> tokenSEMICOLON >> tokenBODY >> tokenDATA >> (- declaration) >> tokenSEMICOLON >> *statement >> tokenEND;</code>	<code>program_rule = BOUNDARIES >> tokenNAME >> program_name >> tokenSEMICOLON >> tokenBODY >> tokenDATA >> declaration__optional >> tokenSEMICOLON >> statement__iteration >> tokenEND;</code>	<pre>{LA_IS,{T_NAME_0},{ "program_rule",{\ {LA_IS,(""),9,{T_NAME_0,"program_name", T_SEMICOLON_0,T_BODY_0,T_DATA_0, "declaration__optional",T_SEMICOLON_0, "statement__iteration",T_END_0}}\ }}\}</pre>
	<code>declaration__optional = declaration "";</code>	<code>declaration__optional → declaration declaration__optional → ε</code>	<code>declaration__optional(1: "INTEGER16") → declaration declaration__optional(1: !"INTEGER16") → ε</code>		<code>declaration__optional = declaration "";</code>	<pre>{LA_IS,{T_DATA_TYPE_0},{ "declaration__optional",{\ {LA_IS,(""),1,{ "declaration" }}\ }}\} {LA_NOT,{T_DATA_TYPE_0},{ "declaration__optional",{\ {LA_IS,(""),0,{ "" }}\ }}\}</pre>
<code>value = [sign] , unsigned_value;</code>	<code>value = sign__optional, unsigned_value;</code>	<code>value → sign__optional unsigned_value</code>	<code>value(1: "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "+", "-") → sign__optional unsigned_value</code>	<code>value = -sign >> unsigned_value >> BOUNDARIES;</code>	<code>value = sign__optional >> unsigned_value >> BOUNDARIES;</code>	

				BOUNDARY = BOUNDARY__SPACE BOUNDARY__TAB BOUNDARY__VERTICAL_TAB BOUNDARY__FORM_FEED BOUNDARY__CARRIAGE_RETURN BOUNDARY__LINE_FEED BOUNDARY__NULL; BOUNDARY__SPACE = " "; BOUNDARY__TAB = "\t"; BOUNDARY__VERTICAL_TAB = "\v"; BOUNDARY__FORM_FEED = "\f"; BOUNDARY__CARRIAGE_RETURN = "\r"; BOUNDARY__LINE_FEED = "\n"; BOUNDARY__NULL = "\0"; NO_BOUNDARY = ""; #define WHITESPACES \ STRICT_BOUNDARIES, \ BOUNDARIES, \ BOUNDARY, \ BOUNDARY__SPACE, \ BOUNDARY__TAB, \ BOUNDARY__VERTICAL_TAB, \ BOUNDARY__FORM_FEED, \ BOUNDARY__CARRIAGE_RETURN, \ BOUNDARY__LINE_FEED, \ BOUNDARY__NULL, \ NO_BOUNDARY	BOUNDARY = BOUNDARY__SPACE BOUNDARY__TAB BOUNDARY__VERTICAL_TAB BOUNDARY__FORM_FEED BOUNDARY__CARRIAGE_RETURN BOUNDARY__LINE_FEED BOUNDARY__NULL; BOUNDARY__SPACE = " "; BOUNDARY__TAB = "\t"; BOUNDARY__VERTICAL_TAB = "\v"; BOUNDARY__FORM_FEED = "\f"; BOUNDARY__CARRIAGE_RETURN = "\r"; BOUNDARY__LINE_FEED = "\n"; BOUNDARY__NULL = "\0"; NO_BOUNDARY = ""; #define WHITESPACES \ STRICT_BOUNDARIES, \ BOUNDARIES, \ BOUNDARY, \ BOUNDARY__SPACE, \ BOUNDARY__TAB, \ BOUNDARY__VERTICAL_TAB, \ BOUNDARY__FORM_FEED, \ BOUNDARY__CARRIAGE_RETURN, \ BOUNDARY__LINE_FEED, \ BOUNDARY__NULL, \ NO_BOUNDARY	
						 \ { LA_IS, { T_NAME_0 }, { "program____part1", \ LA_IS, { "" }, 7, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, T_DATA_0, "declaration_optional", T_SEMICOLON_0 }} \ }}}; \ }; \ "program_rule"